
P'a Comer

Sistema de Reservas

Disciplina: Programação Python

Flávio Saldanha

1. Introdução

O presente relatório descreve o desenvolvimento de um **Sistema de Reservas em Restaurantes**, realizado no âmbito da disciplina de Programação Python. O projeto foi implementado integralmente em Python, sem recurso a frameworks ou bibliotecas externas, e tem como objetivo simular o funcionamento de uma plataforma de reservas semelhante a aplicações reais do setor da restauração.

Todos os dados são guardados localmente num ficheiro no formato JSON, garantindo a persistência da informação entre sessões. A interação com o utilizador é feita exclusivamente através do terminal, com menus de texto simples e validação de entradas.

1.1 Objetivos do Projeto

- Desenvolver um programa funcional em Python aplicando os conceitos fundamentais de programação;
- Implementar estruturas de dados como listas e dicionários para representar reservas e restaurantes;
- Garantir a persistência de dados através de leitura e escrita em ficheiros JSON;
- Validar as entradas do utilizador de forma robusta, evitando erros em tempo de execução;
- Organizar o código em funções reutilizáveis com responsabilidades bem definidas.

1.2 Tecnologias Utilizadas

Linguagem	Python 3.12
Armazenamento	JSON (ficheiro local reservas.json)
Interface	Terminal de texto (linha de comandos)
Bibliotecas	json, os, random, string, datetime
Ficheiro único	reservas.py — sem dependências externas

2. Descrição das Funcionalidades

O programa disponibiliza um menu principal com sete opções, cada uma correspondendo a uma funcionalidade distinta. Ao abrir o programa, é apresentado automaticamente um lembrete com as reservas confirmadas para o dia atual, antes de o menu ser exibido.

Opção	Funcionalidade	Descrição
1	Fazer reserva	Escolhe restaurante, data, hora e pessoas — gera código único.
2	Consultar reserva	Mostra todos os detalhes de uma reserva pelo código.
3	Cancelar reserva	Altera o estado para 'cancelada', mantendo o histórico.
4	Histórico	Lista todas as reservas com filtro opcional por email.
5	Avaliar	Regista uma avaliação de 1 a 5 estrelas numa reserva.
6	Relatório	Estatísticas gerais: restaurante popular, horário de ponta, etc.
0	Sair	Termina o programa.

2.1 Fluxo de uma Reserva

Quando o utilizador seleciona a opção **Fazer Reserva**, o programa percorre os seguintes passos:

1. Apresenta a lista de restaurantes disponíveis com nome, tipo de cozinha e cidade;
2. Solicita a escolha do restaurante, a data (DD/MM/AAAA), a hora (HH:MM) e o número de pessoas;
3. Verifica a disponibilidade — se não houver lugares suficientes, informa o utilizador;
4. Recolhe o nome e email do cliente e mostra um resumo para confirmação;
5. Gera um código único (ex: SQT5245) e guarda a reserva no ficheiro JSON;
6. Apresenta o código de confirmação ao utilizador.

2.2 Estrutura dos Dados

Cada reserva é guardada como um dicionário Python com os seguintes campos:

```
{ "codigo": "SQT5245", "restaurante": "Tasca do Ze",  
  "data": "25/12/2025", "hora": "20:00", "pessoas": 2,  
  "nome": "Joao Silva", "email": "joao@email.com",  
  "estado": "confirmada", "avaliacao": null }
```

3. Dificuldades Encontradas

Ao longo do desenvolvimento do projeto surgiram várias dificuldades técnicas, sobretudo relacionadas com a validação de dados e a gestão de disponibilidade. A tabela seguinte resume os principais obstáculos e a forma como foram resolvidos.

Dificuldade	Como foi resolvida
Validar datas e horas introduzidas pelo utilizador	Uso de try/except com datetime.strptime() para detetar formatos inválidos
Garantir que o código de reserva é único.	Verificação com set() antes de guardar
Erro de digitação enconding -> ecoding	Corrigido na função
MESAS vs Mesas	Corrigido para usar sempre MESAS em maiúsculas
Reservas sem campo 'restaurante'	Apagar reservas.json e criar novas reservas

4. Conclusão

O desenvolvimento deste projeto permitiu consolidar os conceitos fundamentais de programação em Python abordados ao longo deste curso. A implementação de um sistema completo — desde a recolha e validação de dados até à persistência em ficheiro — exigiu a aplicação prática de estruturas de dados, funções, ciclos, condicionais e tratamento de erros.

O resultado final é um programa funcional, organizado e de fácil utilização, que cumpre todos os objetivos definidos inicialmente. O código encontra-se estruturado em funções com responsabilidades bem delimitadas, facilitando a leitura, a manutenção e a expansão futura.

Em retrospectiva, as principais aprendizagens deste trabalho foram a importância de validar sempre os dados introduzidos pelo utilizador, a utilidade das funções auxiliares para evitar repetição de código, e a simplicidade do formato JSON como solução de armazenamento para projetos de pequena dimensão.

